

```

/**
 * Session-aware Logging Utility
 *
 * Provides a standardized way to log DML, errors, and success states with
 * ANSI color support and environmental filtering.
 *
 * @module LogUtil
 * @file log.util.ts
 *
 * Acknowledgments: Architecture for session-reactive logging and atomic DML utility
 * developed in collaboration with AI on Google Search, powered by the Gemini family of models.
 *
 * @copyright 2018-2026, Dennis Jorgenson
 */
// src/loader.ts
import type { ILogger } from "../lib/log.util.js";

import { readdir, readFile } from "node:fs/promises";
import { fileURLToPath } from "node:url";
import { hashKey, hexify } from "#lib/crypto.util";
import { SC } from "#app/lexicon";
import { Load } from "#db/query.util";

/**
 * Manifest Structure Interface
 */
interface ISeedManifest {
  pkey: string;
  keylen?: number;
  resolver?: "hashkey" | "hexify" | "none";
  data: any[];
}

/**
 * Scans and synchronizes JSON-based state manifests into the local database.
 */
export const Loader = async (seedFilePath = "/", context = "Seed", log: ILogger): Promise<void> => {
  const currentContext = `Loader.${context}`;

  // Resolve absolute path for Node 22/23 compatibility
  const seedDirURL = new URL(seedFilePath, import.meta.url);
  const seedDirPath = fileURLToPath(seedDirURL);

  try {
    const files = await readdir(seedDirPath);
    const manifests = files.filter((f) => f.endsWith(".json")).sort();

    log?.info(context, `Initializing System Hydration [${manifests.length} manifests]`);

    for (const fileName of manifests) {
      const filePath = new URL(`.${seedFilePath}${fileName}`, import.meta.url);
      const rawData = await readFile(filePath, "utf8");
      const fileData: Record<string, ISeedManifest> = JSON.parse(rawData);

      for (const [tableName, manifest] of Object.entries(fileData)) {
        const { pkey, keylen, resolver, data } = manifest;

        // Transformation Layer: Handle binary keys and hashes
        const hydrated = data.map((row: any) => {
          // Case 1: Missing field + hashkey resolver = Generate new BINARY(32/16)
          if (resolver === "hashkey") {
            row[pkey] = row[pkey] ? hexify(row[pkey]) : hashKey(keylen || 32);
          }
          // Case 2: Field exists + hexify = Convert string to Buffer
          else if (resolver === "hexify" && row[pkey]) {
            row[pkey] = hexify(row[pkey]);
          }
          return row;
        });

        // Loading Layer: Atomic UPSERT/INSERT
        const result = await Load(hydrated, tableName, currentContext, log);

        if (result.rows > 0) {
          log?.success(context, `[${fileName}] -> ${tableName}: ${result.rows} records added`, SC.OK);
        } else {
          log?.info(context, `[${fileName}] -> ${tableName}: Verified (No changes)`);
        }
      }
    }
  } catch (error) {
    const msg = error instanceof Error ? error.message : "FS_SCAN_ERROR";
    log?.error(context, `Hydration Failed: ${msg}`, SC.UPSERT_FAIL);
  }
};

```